

Báo cáo thiết kế vật lý CPU 4-bit

OpenLane 2 – Sky130 – Core utilization 65%

Sinh viên: _____

Ngày 26 tháng 7 năm 2026

Mục lục

1	Mục tiêu báo cáo	3
2	Lý do chọn thiết kế CPU 4-bit	3
3	Công cụ và công nghệ	3
4	Cấu hình thiết kế	4
4.1	Giải thích các tham số	4
5	Lý do chọn core utilization 65%	5
6	Pin placement	5
7	Quy trình physical design điều khiển bằng Tcl	5
7.1	Kiểm tra RTL	5
7.2	Kết quả synthesis đầu vào	6
7.3	Mở OpenROAD GUI	6
7.4	Đọc PDK, thư viện và netlist synthesis	6
7.5	Khởi tạo floorplan 65%	7
7.6	Đặt I/O pin	7
7.7	Tapcell và well-tie	8
7.8	Power planning	8
7.9	Global placement và detailed placement	9
7.10	Clock-tree synthesis	9
7.11	Routing	10
7.12	Signoff	10
8	Sự cố LVS trong flow thủ công và cách khắc phục	11
8.1	Hiện tượng và phương pháp khoanh vùng	11
8.2	Lệnh cũ gây ra lỗi	11
8.3	Thứ tự lệnh mới	12
8.4	Sửa powered Verilog dùng cho LVS	12
9	Signoff và mô phỏng post-layout của layout đã sửa	12
9.1	Stream-out GDS và DRC	12
9.2	RC parasitic extraction và post-layout STA	13
9.3	Tạo SDF và gate-level simulation	13

10 Kết quả tham chiếu của thiết kế 65%	14
11 Phân tích kết quả	14
12 Các lưu ý kỹ thuật	14
13 Kết luận	15

1 Mục tiêu báo cáo

Báo cáo trình bày quá trình chuyển thiết kế CPU 4-bit từ RTL SystemVerilog sang layout GDSII bằng OpenLane 2. Thiết kế sử dụng công nghệ SkyWater 130 nm, thư viện standard cell `sky130_fd_sc_hd`, chu kỳ clock 20 ns và mục tiêu core utilization bằng 65%.

Mục tiêu chính gồm:

- kiểm tra chức năng RTL trước khi triển khai vật lý;
- tổng hợp RTL thành netlist standard cell;
- xây dựng floorplan với utilization 65%;
- thực hiện placement, clock-tree synthesis và routing;
- kiểm tra timing, DRC, antenna và LVS;
- giải thích lý do lựa chọn từng tham số quan trọng.

2 Lý do chọn thiết kế CPU 4-bit

CPU 4-bit được chọn vì có đầy đủ những thành phần cơ bản của một hệ thống số tuần tự nhưng vẫn đủ nhỏ để quan sát rõ từng giai đoạn physical design. Thiết kế có datapath, ALU, thanh ghi, bộ đếm chương trình, bộ điều khiển trạng thái và đường clock. Vì vậy, nó phù hợp để minh họa toàn bộ flow RTL-to-GDSII mà không làm thời gian chạy và việc phân tích trở nên quá phức tạp.

Các khối RTL chính là:

- `cpu_4bit.sv`: top-level, chứa PC, IR, accumulator và các cờ;
- `alu_4bit.sv`: thực hiện cộng, trừ, logic, dịch, tăng và giảm;
- `control_unit.sv`: FSM gồm FETCH, EXECUTE và HALT;
- program memory đặt bên ngoài CPU và không được tổng hợp vào core.

Giao tiếp top-level gồm clock, reset, bus địa chỉ instruction 4-bit, bus dữ liệu instruction 8-bit và tín hiệu `halted`. Việc tách program memory ra khỏi core giúp phạm vi physical design tập trung vào logic CPU.

3 Công cụ và công nghệ

Thành phần	Lựa chọn và lý do
OpenLane 2	Điều phối flow RTL-to-GDSII và lưu state sau từng bước, thuận tiện cho việc dừng, kiểm tra và chạy tiếp.
OpenROAD	Thực hiện floorplan, placement, CTS, routing và phân tích vật lý.
Yosys	Tổng hợp RTL và ánh xạ logic sang standard cell.
Sky130A	PDK mã nguồn mở, phù hợp cho học tập và nghiên cứu ASIC.
<code>sky130_fd_sc_hd</code>	Thư viện high-density giúp tạo layout nhỏ gọn cho CPU có diện tích logic nhỏ.
KLayout/Magic	Kiểm tra và quan sát layout; hỗ trợ kiểm tra DRC và stream-out.

4 Cấu hình thiết kế

File `config_util165.json` lưu các tham số gốc của thiết kế 65%. Trong quy trình thủ công, file này chỉ là tài liệu tham chiếu; các giá trị tương ứng được truyền bằng Tcl thay vì đưa file cho một OpenLane flow:

Listing 1: Cấu hình OpenLane 2

```
1 {
2   "meta": {
3     "version": 2,
4     "flow": "Classic"
5   },
6
7   "PDK": "sky130A",
8   "STD_CELL_LIBRARY": "sky130_fd_sc_hd",
9
10  "DESIGN_NAME": "cpu_4bit",
11  "VERILOG_FILES": [
12    "dir::alu_4bit.sv",
13    "dir::control_unit.sv",
14    "dir::cpu_4bit.sv"
15  ],
16
17  "CLOCK_PORT": "clk",
18  "CLOCK_PERIOD": 20.0,
19
20  "FP_CORE_UTIL": 65,
21  "FP_PIN_ORDER_CFG": "dir::pin_order.cfg",
22  "ERRORS_ON_UNMATCHED_IO": "both"
23 }
```

4.1 Giải thích các tham số

Tham số	Ý nghĩa và lý do lựa chọn
flow: Classic	Chọn flow tiêu chuẩn từ RTL đến GDSII, phù hợp với thiết kế standard-cell không có macro.
PDK: sky130A	Chọn bộ quy tắc và model công nghệ SkyWater 130 nm.
sky130_fd_sc_hd	Chọn standard-cell library high-density để giảm diện tích.
DESIGN_NAME	Phải trùng với module top-level <code>cpu_4bit</code> .
VERILOG_FILES	Chỉ chứa RTL tổng hợp được; testbench và program memory mô phỏng không được đưa vào synthesis.
CLOCK_PORT: clk	Xác định clock chính cho STA và clock-tree synthesis.
CLOCK_PERIOD: 20.0	Chu kỳ 20 ns tương đương tần số mục tiêu $f = 1/T = 50$ MHz. Đây là mục tiêu vừa phải đối với CPU nhỏ trên Sky130.
FP_CORE_UTIL: 65	Yêu cầu diện tích cell ban đầu chiếm khoảng 65% vùng core khi OpenLane tính kích thước floorplan.

Tham số	Ý nghĩa và lý do lựa chọn
FP_PIN_ORDER_CFG	Điều khiển cạnh và thứ tự I/O để kết nối logic rõ ràng và routing thuận lợi.
ERRORS_ON_UNMATCHED_IO	Dừng flow nếu danh sách pin không khớp port RTL, tránh tạo layout thiếu hoặc sai chân.

5 Lý do chọn core utilization 65%

Core utilization được hiểu gần đúng là:

$$U = \frac{A_{\text{cell}}}{A_{\text{core}}} \times 100\%.$$

Mục tiêu 65% tạo ra sự cân bằng giữa diện tích và khoảng trống triển khai. Nếu utilization quá thấp, core lớn hơn cần thiết, làm tăng diện tích die và có thể làm dây dài hơn. Nếu utilization quá cao, các cell bị ép sát nhau, không còn đủ không gian cho buffer, clock tree, tối ưu timing và routing. Điều đó có thể gây congestion và DRC violation.

Giá trị 65% được chọn vì thiết kế CPU này nhỏ, không chứa macro và kết quả signoff cho thấy placement và routing vẫn hoàn thành sạch. Đây là giá trị mục tiêu của bước floorplan, không nhất thiết bằng utilization cuối cùng. Sau CTS, timing repair và các bước chèn cell, số lượng và diện tích cell có thể thay đổi.

6 Pin placement

Các bus instruction được bố trí theo hướng luồng dữ liệu:

- `imem_addr[3:0]` ở cạnh Bắc;
- `imem_rdata[7:0]` ở cạnh Nam;
- `clk` và `rst_n` ở cạnh Tây;
- `halted` ở cạnh Đông.

Cách bố trí này nhóm các tín hiệu cùng chức năng, làm sơ đồ dễ đọc và giảm khả năng các bus giao cắt không cần thiết. OpenLane vẫn căn tọa độ pin vào routing grid hợp lệ.

7 Quy trình physical design điều khiển bằng Tcl

Trong phương pháp này, OpenLane không được dùng để chạy từng flow step. OpenROAD được mở trực tiếp và toàn bộ thao tác physical design được gọi từ Tcl console. OpenLane chỉ cung cấp Nix shell chứa đúng phiên bản tool và PDK. Sau mỗi công đoạn, lệnh `write_db` tạo checkpoint ODB để có thể đóng GUI và tiếp tục sau đó.

7.1 Kiểm tra RTL

Mô phỏng chức năng phải hoàn thành trước physical design. Physical design không sửa được lỗi thuật toán hoặc lỗi FSM trong RTL. Kết quả mong đợi của testbench là:

```
1 TEST PASSED: ACC = 6
```

7.2 Kết quả synthesis đầu vào

Yosys đã đọc ba file RTL, elaboration thiết kế, tối ưu logic và ánh xạ sang cell của sky130_fd_sc_hd. Physical design thử công không chạy lại synthesis mà sử dụng trực tiếp gate-level netlist đã tồn tại:

```
1 runs/manual_pd_01/06-yosys-synthesis/cpu_4bit.nl.v
```

Sau synthesis cần kiểm tra:

- không có latch ngoài ý muốn;
- không còn logic chưa ánh xạ;
- top module đúng là `cpu_4bit`;
- netlist không chứa testbench hoặc program ROM;
- clock và reset vẫn được nối đúng.

7.3 Mở OpenROAD GUI

Mở một GUI trống từ thư mục project:

```
1 cd /home/minhuy/Workspaces/cpu_4bit
2
3 nix-shell /home/minhuy/EDA/openlane2/shell.nix \
4   --run 'openroad -gui'
```

Không dùng `openlane -from`, `openlane -to` hoặc một flow step nào. Các lệnh ở những phần tiếp theo được nhập trực tiếp vào Tcl console của OpenROAD.

7.4 Đọc PDK, thư viện và netlist synthesis

Đầu tiên khai báo đường dẫn PDK, sau đó đọc technology LEF, cell LEF, Liberty và synthesized netlist:

```
1 set run_dir runs/manual_tcl_util65
2
3 set pdk /home/minhuy/.volare/volare/sky130/versions/\
4 0fe599b2afb6708d281543108caf8310912f54af/sky130A
5
6 read_lef $pdk/libs.ref/sky130_fd_sc_hd/techlef/\
7 sky130_fd_sc_hd__nom.tlef
8 read_lef $pdk/libs.ref/sky130_fd_sc_hd/lef/\
9 sky130_fd_sc_hd.lef
10 read_lef $pdk/libs.ref/sky130_fd_sc_hd/lef/\
11 sky130_ef_sc_hd.lef
12
13 read_liberty $pdk/libs.ref/sky130_fd_sc_hd/lib/\
14 sky130_fd_sc_hd__tt_025C_1v80.lib
15
16 read_verilog runs/manual_pd_01/06-yosys-synthesis/\
17 cpu_4bit.nl.v
18 link_design cpu_4bit
19
20 create_clock -name clk -period 20.0 [get_ports clk]
21 write_db $run_dir/checkpoint_00_synthesis_loaded.odb
```

```

22
23 sta::redirect_file_begin $run_dir/checkpoint_00_report.rpt
24 report_clock_properties
25 report_design_area
26 sta::redirect_file_end

```

Lệnh `link_design` tạo design database từ netlist. Lệnh `create_clock` biểu diễn constraint 20 ns để OpenROAD có thể phân tích timing trong các bước placement và CTS. Clock chưa được đặt ở chế độ propagated trước CTS; giai đoạn này dùng ideal clock. Binary OpenROAD đang sử dụng không có lệnh `redirect` hoặc `tee`, vì vậy báo cáo được lưu bằng cặp lệnh `sta::redirect_file_begin` và `sta::redirect_file_end`.

7.5 Khởi tạo floorplan 65%

Floorplan xác định die, core và placement rows. Lệnh Tcl sử dụng là:

```

1 initialize_floorplan -utilization 65 -aspect_ratio 1.0 -core_space {10.88 10.88 5
  .52 5.52} -site unithd
2 make_tracks
3
4 write_db $run_dir/checkpoint_01_floorplan.odb
5 write_def $run_dir/checkpoint_01_floorplan.def

```

`-aspect_ratio 1.0` yêu cầu core gần hình vuông. Hình dạng này phù hợp với một CPU nhỏ không chứa macro và không có yêu cầu package đặc biệt.

Thứ tự là `bottom top left right`. Placement site `unithd` có chiều cao $2.72\ \mu\text{m}$ và chiều rộng $0.46\ \mu\text{m}$. OpenLane dùng bốn lần chiều cao site cho biên trên/dưới và mười hai lần chiều rộng site cho biên trái/phải:

$$M_{\text{bottom}} = M_{\text{top}} = 4 \times 2.72 = 10.88\ \mu\text{m},$$

$$M_{\text{left}} = M_{\text{right}} = 12 \times 0.46 = 5.52\ \mu\text{m}.$$

Các khoảng trống này dành cho I/O, power distribution và routing gần biên. Việc sử dụng bội số nguyên của placement site cũng giữ core thẳng hàng với standard-cell grid.

`initialize_floorplan` chỉ tạo die, core và placement rows. Lệnh `make_tracks` phải được gọi sau đó để tạo routing tracks từ pitch và offset trong technology LEF. Nếu thiếu bước này, `place_pins` báo lỗi PPL-0021 vì không tìm thấy horizontal track trên `met3`.

7.6 Đặt I/O pin

Đặt các pin lên layer ngang `met3` và layer dọc `met2`:

```

1 set_io_pin_constraint -pin_names {imem_addr[3] imem_addr[2] imem_addr[1]
  imem_addr[0]} -region top:* -group -order
2 set_io_pin_constraint -pin_names {imem_rdata[7] imem_rdata[6] imem_rdata[5]
  imem_rdata[4] imem_rdata[3] imem_rdata[2] imem_rdata[1] imem_rdata[0]} -region
  bottom:* -group -order
3 set_io_pin_constraint -pin_names {clk rst_n} -region left:* -group -order
4 set_io_pin_constraint -pin_names {halted} -region right:*
5 place_pins -hor_layers met3 -ver_layers met2 -corner_avoidance 5
  -min_distance_in_tracks
6
7 set block [ord::get_db_block]

```

```

8 set pins_without_geometry {}
9 foreach bterm [$block getBTerms] {
10     if {[llength [$bterm getBPins]] == 0} {
11         lappend pins_without_geometry [$bterm getName]
12     }
13 }
14 puts "Pins without geometry: $pins_without_geometry"
15
16 write_db $run_dir/checkpoint_02_pins.odb
17 write_def $run_dir/checkpoint_02_pins.def

```

Trong Tcl API, `top`, `bottom`, `left` và `right` tương ứng Bắc, Nam, Tây và Đông. Dấu ngoặc vuông trong tên bus được bảo vệ bởi Tcl braces. Các lớp `met2/met3` được chọn vì chúng nằm trong routing stack hợp lệ của Sky130 và cho phép pin kết nối thuận lợi với router.

7.7 Tapcell và well-tie

Tapcell nối well/substrate về nguồn để tránh well bị floating và giảm nguy cơ latch-up. Khoảng cách mục tiêu là $13\mu\text{m}$:

```

1 tapcell -distance 13 -tapcell_master sky130_fd_sc_hd__tapvpwrvgnd_1
   -endcap_master sky130_fd_sc_hd__decap_3 -halo_width_x 10 -halo_width_y 10
2 write_db $run_dir/checkpoint_03_tapcells.odb
3 write_def $run_dir/checkpoint_03_tapcells.def

```

Kết quả kiểm tra thực tế là 30 tapcell, 36 endcap và tổng cộng 206 instances. Ngay sau tapcell, revision OpenROAD được sử dụng đã in Design area 0 kèm 68% utilization. Vì hai trường trong cùng dòng không nhất quán, diện tích movable cell được kiểm tra trực tiếp từ kích thước master trong OpenDB. Kết quả là 140 movable instances với diện tích $1591.5264\mu\text{m}^2$, xác nhận netlist logic không bị mất.

7.8 Power planning

Power planning tạo mạng phân phối VPWR và VGND. Mục tiêu là đưa nguồn đến các hàng standard cell bằng đường kim loại có điện trở đủ thấp. Sau khi tạo PDN cần kiểm tra connectivity và IR drop. Phân tích IR trong flow này vẫn mang tính ước lượng vì không có file vị trí nguồn ngoài VSRC_LOC_FILES.

```

1 add_global_connection -net VPWR -inst_pattern .* -pin_pattern {^VPWR$} -power
2 add_global_connection -net VPWR -inst_pattern .* -pin_pattern {^VPB$} -power
3 add_global_connection -net VGND -inst_pattern .* -pin_pattern {^VGND$} -ground
4 add_global_connection -net VGND -inst_pattern .* -pin_pattern {^VNB$} -ground
5 global_connect
6
7 set_voltage_domain -name CORE -power VPWR -ground VGND
8 define_pdn_grid -name stdcell_grid -starts_with POWER -voltage_domain CORE -pins
   {met4 met5}
9
10 add_pdn_stripe -grid stdcell_grid -layer met1 -width 0.48 -followpins
   -starts_with POWER
11 add_pdn_stripe -grid stdcell_grid -layer met4 -width 1.6 -pitch 153.6 -offset 16
   .32 -spacing 1.7 -starts_with POWER

```



```

12 add_pdn_stripe -grid stdcell_grid -layer met5 -width 1.6 -pitch 153.18 -offset 16
    .65 -spacing 1.7 -starts_with POWER
13
14 add_pdn_connect -grid stdcell_grid -layers {met1 met4}
15 add_pdn_connect -grid stdcell_grid -layers {met4 met5}
16
17 pdngen
18 check_power_grid -net VPWR -error_file $run_dir/VPWR_grid_errors.rpt
19 check_power_grid -net VGND -error_file $run_dir/VGND_grid_errors.rpt
20 write_db $run_dir/checkpoint_04_pdn.odt
21 write_def $run_dir/checkpoint_04_pdn.def

```

Các giá trị width, pitch và offset phải thỏa design rules của PDK. Trước khi dùng cho tape-out, cần xác nhận lại chúng bằng DRC và power-integrity analysis; các giá trị trên phù hợp cho thực hành với core nhỏ.

7.9 Global placement và detailed placement

Global placement phân bố cell để tối ưu tổng wirelength và congestion. Detailed placement căn từng cell vào row và site hợp lệ, đồng thời loại bỏ overlap. Ở utilization 65%, placement phải dành đủ không gian cho cell được thêm vào ở các bước sửa timing và CTS.

```

1 set_wire_rc -signal -layer met2
2 set_wire_rc -clock -layer met3
3
4 global_placement -density 0.75 -timing_driven -routability_driven -pad_left 0
    -pad_right 0 -init_wirelength_coef 0.25
5 detailed_placement
6 optimize_mirroring
7 check_placement -verbose
8
9 write_db $run_dir/checkpoint_05_placement.odt
10 write_def $run_dir/checkpoint_05_placement.def

```

Trong lần chạy đầu tiên chưa có metrics của một implementation trước đó. Vì vậy, target global placement ban đầu được chọn bằng heuristic:

$$D_{GPL} = U_{\text{floorplan}} + 5P_{\text{cell}} + 10.$$

Với $U_{\text{floorplan}} = 65\%$ và cell padding $P_{\text{cell}} = 0$, target đầu tiên là 75%, tức tham số Tcl bằng 0.75. Phần chênh 10 điểm phần trăm là headroom cho thuật toán global placement. Sau lần chạy đầu, congestion, overflow và timing report mới được dùng để quyết định giữ hay điều chỉnh density.

7.10 Clock-tree synthesis

CTS chèn buffer và xây dựng cây clock nhằm giảm skew, kiểm soát insertion delay và giới hạn tải trên đường clock. Clock không nên được routing như một net tín hiệu thông thường vì nó điều khiển đồng thời nhiều flip-flop.

```

1 unset_propagated_clock [all_clocks]
2

```

```

3 clock_tree_synthesis -root_buf sky130_fd_sc_hd__clkbuf_16 -buf_list {
    sky130_fd_sc_hd__clkbuf_8 sky130_fd_sc_hd__clkbuf_4 sky130_fd_sc_hd__clkbuf_2}
    -sink_clustering_size 25 -sink_clustering_max_diameter 50
    -sink_clustering_enable
4
5 set_propagated_clock [all_clocks]
6 detailed_placement
7 check_placement -verbose
8 write_db $run_dir/checkpoint_06_cts.odb
9 write_def $run_dir/checkpoint_06_cts.def

```

7.11 Routing

Global routing ước lượng đường đi và congestion. Detailed routing tạo dây và via thật trên các lớp kim loại theo design rules của Sky130. Router có thể lặp lại nhiều lần để sửa short, spacing và các vi phạm hình học khác.

```

1 set_routing_layers -signal met1-met5
2 set_routing_layers -clock met1-met5
3 set_global_routing_layer_adjustment * 0.30
4
5 global_route -congestion_iterations 50 -verbose
6
7 detailed_route -bottom_routing_layer met1 -top_routing_layer met5 -output_drc
    $run_dir/detailed_route_drc.rpt -droute_end_iter 64 -or_seed 42 -verbose 1
8
9 write_db $run_dir/checkpoint_07_routed.odb
10 write_def $run_dir/checkpoint_07_routed.def

```

Trong lần chạy đầu không dùng `-allow_congestion`. Nếu global router còn overflow, cần dừng để điều chỉnh floorplan, placement hoặc routing resources trước khi detailed routing.

7.12 Signoff

Các kiểm tra cuối gồm:

- STA setup/hold ở nhiều corner;
- DRC bằng router, Magic và KLayout;
- antenna check;
- LVS so sánh layout với netlist;
- XOR check giữa các biểu diễn layout;
- power và IR-drop estimation.

```

1 report_checks -path_delay min_max -format full_clock_expanded
2 report_worst_slack -max
3 report_worst_slack -min
4 report_tns
5
6 check_antennas
7 check_power_grid -net VPWR
8 check_power_grid -net VGND
9

```

```

10 write_verilog -include_pwr_gnd $run_dir/cpu_4bit_manual_final.v
11 write_db $run_dir/cpu_4bit_manual_final.odb
12 write_def $run_dir/cpu_4bit_manual_final.def

```

OpenROAD cung cấp timing, routing và antenna checks. Signoff DRC bằng Magic/KLayout, LVS và GDS stream-out cần các Tcl/script tương ứng của từng tool; chúng không được thay thế chỉ bằng việc lưu ODB.

8 Sự cố LVS trong flow thủ công và cách khắc phục

8.1 Hiện tượng và phương pháp khoanh vùng

Ở lần chạy thủ công đầu tiên, detailed routing hoàn thành và OpenROAD không báo routing DRC. Tuy nhiên, Magic extraction báo 10 lỗi:

```

1 Illegal overlap between obsli1 and viali (types do not connect)

```

Netgen sau đó cho thấy số lượng device của layout và schematic bằng nhau, nhưng connectivity không khớp. Vì vậy lỗi không nằm ở synthesis hay số lượng standard cell mà nằm ở kết nối hình học sau routing.

Các checkpoint được kiểm tra lần lượt bằng cùng script `Magic.SpiceExtraction` của OpenLane:

Checkpoint	Magic extraction
checkpoint_07_cts.def	0 lỗi
checkpoint_09_detailed_routing.def	0 lỗi
checkpoint_10_filler.def	10 lỗi

Kết quả này xác định filler insertion là thời điểm đầu tiên tạo lỗi. Các cell decap/filler được chèn vào khoảng trống sau khi dây đã được route. Một số obstruction `obsli1` của các cell mới chèn chồng lên `li1/viali` đã tồn tại.

8.2 Lệnh cũ gây ra lỗi

Thứ tự cũ là routing trước, filler insertion sau:

```

1 read_db $run_dir/checkpoint_07_cts.odb
2 global_route -congestion_iterations 50 -verbose
3 detailed_route -bottom_routing_layer met1 -top_routing_layer met5 -output_drc
   $run_dir/detailed_route_drc.rpt -droute_end_iter 64 -or_seed 42 -verbose 1
4 write_db $run_dir/checkpoint_09_detailed_routing.odb
5 write_def $run_dir/checkpoint_09_detailed_routing.def
6
7 filler_placement -prefix FILLER_ {sky130_ef_sc_hd__decap_12
   sky130_fd_sc_hd__decap_8 sky130_fd_sc_hd__decap_6 sky130_fd_sc_hd__decap_4
   sky130_fd_sc_hd__decap_3 sky130_fd_sc_hd__fill*}
8 write_db $run_dir/checkpoint_10_filler.odb
9 write_def $run_dir/checkpoint_10_filler.def

```

Thử chạy lại `detailed_route` trực tiếp trên checkpoint 10 không thành công vì database đã chứa routing cũ. Detailed router dừng với `DRT-1010 Unsupported non-orthogonal wire`. Do đó không thể sửa bằng cách gọi lại detailed routing trực tiếp trên checkpoint này.

8.3 Thứ tự lệnh mới

Giải pháp là quay về checkpoint CTS chưa có signal routing, chèn filler trước, rồi chạy global và detailed routing. Khi đó router nhìn thấy obstruction của toàn bộ decap/filler và tránh chúng ngay từ đầu.

```
1 read_db $run_dir/checkpoint_07_cts.odb
2 filler_placement -prefix FILLER_ {sky130_ef_sc_hd__decap_12
   sky130_fd_sc_hd__decap_8 sky130_fd_sc_hd__decap_6 sky130_fd_sc_hd__decap_4
   sky130_fd_sc_hd__decap_3 sky130_fd_sc_hd__fill*}
3 global_connect
4 set_routing_layers -signal met1-met5
5 set_routing_layers -clock met1-met5
6 set_global_routing_layer_adjustment * 0.30
7 global_route -congestion_iterations 50 -verbose
8 detailed_route -bottom_routing_layer met1 -top_routing_layer met5 -output_drc
   $run_dir/detailed_route_with_fillers_drc.rpt -droute_end_iter 64 -or_seed 42
   -verbose 1
9 write_db $run_dir/checkpoint_11_fillers_before_routing.odb
10 write_def $run_dir/checkpoint_11_fillers_before_routing.def
```

Magic extraction trên checkpoint 11 hoàn thành mà không có extraction error. Các kiểm tra sau khi sửa cũng cho kết quả antenna bằng 0 và toàn bộ shape trên hai net VPWR, VGND đều connected.

8.4 Sửa powered Verilog dùng cho LVS

Lệnh cũ chỉ ghi logical Verilog:

```
1 write_verilog $run_dir/cpu_4bit_manual_final_fixed.v
```

File tạo ra không có hai top-level power ports và không ghi power/ground pin của các instance. Vì vậy Netgen báo 152 device ở cả hai phía nhưng số net là 156 và 760.

Lệnh đúng cho LVS phải bao gồm power và ground:

```
1 global_connect
2 write_verilog -include_pwr_gnd $run_dir/cpu_4bit_manual_final_fixed.v
```

Powered Verilog mới có 317 kết nối .VPWR(...) tương ứng với 317 instance trong layout. Kết quả LVS cuối cùng là:

```
1 Circuit 1 contains 152 devices, Circuit 2 contains 152 devices.
2 Circuit 1 contains 156 nets,    Circuit 2 contains 156 nets.
3 Final result:
4 Circuits match uniquely.
```

Như vậy, hai thay đổi cần thiết so với flow cũ là chèn filler trước khi routing và dùng `write_verilog -include_pwr_gnd` để tạo schematic netlist cho LVS.

9 Signoff và mô phỏng post-layout của layout đã sửa

9.1 Stream-out GDS và DRC

DEF đã sửa được stream-out lại thành `cpu_4bit_manual_final_fixed.gds`. GDS cũ không được tái sử dụng vì nó không chứa thứ tự filler/routing mới. KLayout chạy deck `sky130A_mr.drc` với các nhóm FEOL, BEOL và off-grid được bật:

```

1 klayout -b -zz -r $pdk/libs.tech/klayout/drc/sky130A_mr.drc -rd input=$PWD/runs/
  manual_tcl_util65/cpu_4bit_manual_final_fixed.gds -rd topcell=cpu_4bit -rd
  report=$PWD/runs/manual_tcl_util65/reports_fixed/drc_violations.klayout.xml -
  rd feol=true -rd beol=true -rd floating_metal=false -rd offgrid=true -rd seal=
  true -rd threads=16

```

Report database cuối chứa 0 DRC item. Cùng với LVS match uniquely, kết quả này xác nhận GDS fixed không có vi phạm được hai signoff deck phát hiện và connectivity layout khớp powered netlist.

9.2 RC parasitic extraction và post-layout STA

OpenRCX dùng nominal rule set của Sky130 để trích xuất điện trở và điện dung ký sinh từ routing:

```

1 read_db $run_dir/cpu_4bit_manual_final_fixed.odb
2 define_process_corner -ext_model_index 0 CURRENT_CORNER
3 extract_parasitics -ext_model_file $pdk/libs.tech/openlane/
  rules.openrcx.sky130A.nom.calibre -lef_res
4 write_spef $run_dir/cpu_4bit_manual_final_fixed.nom.spef

```

Sau khi nạp nominal Liberty, clock 20 ns và SPEF, post-layout STA cho kết quả:

Chỉ tiêu post-layout nominal	Kết quả
Setup worst slack	+16.19 ns
Hold worst slack	+0.53 ns
Total negative slack	0.00 ns

Cả setup và hold slack đều dương, đồng thời TNS bằng 0, nên timing đạt ở nominal corner với clock period 20 ns.

9.3 Tạo SDF và gate-level simulation

SDF dùng cho timing back-annotation được tạo từ trạng thái STA đã nạp SPEF:

```

1 write_sdf -corner nom_tt_025C_1v80 -include_typ -digits 4 $run_dir/
  cpu_4bit_manual_final_fixed.nom.sdf

```

Testbench tb_gls.sv annotate file này vào instance cpu_4bit_tb_gls.dut với chế độ TYPICAL. Xcelium compile powered netlist, Sky130 UDP primitives, timing models và testbench bằng:

```

1 xrun -64bit -sv -define USE_POWER_PINS -timescale 1ns/1ps -access +rwc -
  xmlibdirname runs/manual_tcl_util65/xcelium_postlayout_fixed.d $pdk/libs.ref/
  sky130_fd_sc_hd/verilog/primitives.v gls_models/sky130_fd_sc_hd_reset_width.v
  runs/manual_tcl_util65/cpu_4bit_manual_final_fixed.v tb_gls.sv -top
  cpu_4bit_tb_gls

```

Xcelium báo SDF annotation thành công:

- 722/722 path delays được annotate, đạt 100%;
- 114/114 timing checks được annotate, đạt 100%;

- mô phỏng kết thúc tại 250 ns với thông báo POST-LAYOUT GLS PASSED: ACC = 6.

Do đó flow thủ công fixed đã hoàn thành đủ các mục tiêu: layout, GDS stream-out, DRC, LVS, parasitic extraction, post-layout STA và post-layout gate-level simulation.

10 Kết quả tham chiếu của thiết kế 65%

Các số liệu dưới đây lấy từ implementation 65% đã được kiểm chứng tại runs/manual_pd_util65/final. Chúng là mốc tham chiếu để đánh giá quá trình chạy Tel thủ công. Sau khi hoàn thành manual flow, cần trích xuất lại metrics vì việc thay đổi seed, placement option hoặc PDN có thể làm kết quả khác đi.

Chỉ tiêu	Kết quả
Die area	4311.44 μm^2
Core area	2409.81 μm^2
Final instance utilization	67.7051%
Số instance cuối	196
Tổng instance area	1631.56 μm^2
Setup worst slack	+11.859 ns
Hold worst slack	+0.115 ns
Total routed wirelength	3137 μm
Maximum net wirelength	150.69 μm
Số via	1180
Total power ước lượng	0.4218 mW
Worst IR drop ước lượng	0.166 mV
Final routing DRC violations	0
Antenna violations	0
Magic DRC violations	0
KLayout DRC violations	0
LVS errors	0
XOR differences	0

11 Phân tích kết quả

Utilization cuối bằng 67.7051%, cao hơn mục tiêu floorplan 65%. Đây không phải lỗi. Giá trị 65% được dùng để tính kích thước core tại thời điểm floorplan, còn flow có thể chèn hoặc thay đổi buffer, clock cells, tie cells và cell sửa timing ở những bước sau.

Setup slack và hold slack đều dương, cho thấy thiết kế đáp ứng constraint clock 20 ns trong các corner được kiểm tra. Hold slack nhỏ hơn setup slack, nhưng vẫn dương nên không có hold violation.

Kết quả DRC, antenna và LVS đều bằng 0 cho thấy layout hợp lệ về hình học, không có lỗi antenna được báo và kết nối vật lý khớp với netlist. Worst IR drop ước lượng rất nhỏ so với nguồn 1.8 V. Tuy nhiên, kết luận IR-drop cần được xem là sơ bộ do chưa mô hình hóa chính xác vị trí nguồn package.

12 Các lưu ý kỹ thuật

- Không đồng nhất target utilization với final utilization.

- Không sửa utilization bằng cách thay đổi trực tiếp die sau khi đã placement; nên quay lại checkpoint floorplan hoặc tạo một bộ checkpoint mới.
- GUI vừa dùng để quan sát vừa cung cấp Tcl console. Người thiết kế phải chủ động lưu ODB sau từng công đoạn để flow thủ công có thể lặp lại.
- Synthesis state không chứa DEF hoặc ODB. Vì vậy OpenROAD chưa có layout để hiển thị cho đến khi chạy `initialize_floorplan`.
- Cần giữ đường dẫn PDK và version PDK nhất quán khi chạy thủ công.
- Trước tape-out thực tế cần có nguồn activity chính xác, package model và vị trí nguồn để phân tích power integrity đáng tin cậy hơn.

13 Kết luận

CPU 4-bit đã được triển khai thành công với target core utilization 65% trên Sky130. Flow hoàn thành synthesis, floorplan, placement, CTS, routing và signoff. Timing setup/hold đều đạt, các kiểm tra DRC, antenna và LVS đều pass. Kết quả cho thấy 65% là một target khả thi cho phiên bản RTL hiện tại, đồng thời tạo ra một layout nhỏ gọn mà router vẫn xử lý thành công.